Aetra

decentralized proof-of-stake execution network

Daniil Shcherbakov

July 8, 2026

Abstract

The aim of this text is to provide a first outline of Aetra: a decentralized blockchain network designed for trust, moderate speed, and practical validator operation rather than maximum throughput at any cost. Aetra targets stronger-than-average decentralization, a medium hardware profile, bounded validator influence, and native smart contracts based on the Aetra Virtual Machine.

The system is intended to support proof-of-stake validation, nominator pools, controlled inflation, storage accountability, and future throughput scaling without making synchronization or validator entry unnecessarily hard.

Introduction

Aetra is a blockchain network project built around a conservative trade-off: it should be faster and more usable than slow settlement layers, but it should not sacrifice decentralization, auditability, or validator accessibility in order to compete for the highest possible transaction count.

The validator economy is an important part of this design. Aetra should not let the largest holders grow faster only because they already control more stake. Validator power caps, nominator pools, commission bounds, concentration metrics, and reward modifiers are intended to reduce cartel formation and pressure against a small set of operators.

At the same time, the network must remain economically realistic. Validators should earn enough to justify reliable infrastructure, monitoring, and operational risk, but the protocol should avoid turning high yield into its main product.

Aetra also includes a native virtual machine for smart contracts. Application logic such as tokens, NFTs, domains, markets, and exchange contracts should be implemented through AVM standards, while protocol-critical systems remain native parts of the chain.

Contents

1 Brief Description of Aetra Components

Aetra is intended to be a decentralized proof-of-stake execution network with a small native protocol core and a programmable contract layer. The design does not attempt to make the fastest possible blockchain. Instead, it chooses a more conservative target: practical decentralization, finality fast enough for ordinary applications, and operating requirements that do not exclude independent validators. This chapter gives a brief description of the principal components and design boundaries of the Aetra network.

Aether Core

Aether Core is the coordination layer of the network. It is the part of Aetra that should remain minimal, deterministic, and auditable. It maintains consensus-facing state, validator lifecycle rules, protocol configuration, fee accounting, storage accountability, routing commitments, and upgrade safety. Aether Core should not contain ordinary application business logic. Its purpose is to maintain the rules that all other execution must obey.

The core is built on a BFT proof-of-stake base. The intended network profile targets medium hardware, moderate block times, and finality that is fast enough for normal applications without requiring extreme node resources. These targets are engineering constraints rather than marketing claims. The chain should be faster than slow settlement layers while remaining easier to operate and verify than systems that assume very heavy validator hardware.

Accounts and Address Policy

Aetra contains an account and address layer for ordinary users, validators, contracts, and reserved system entities. This layer should make addresses readable for users while preserving strict validation rules for the protocol. It should prevent malformed addresses, zero-address misuse, and accidental overlap between user accounts and addresses reserved for system responsibilities.

The address model is also part of network clarity. A user-facing address should identify where value or messages are going, while system addresses should clearly represent built-in network duties. This separation reduces ambiguity for wallets, explorers, validators, and applications that need to distinguish user activity from protocol activity.

Validator Economy

Aetra's staking system is designed around validator self-stake, nominator pools, validator registry state, election logic, insurance, delegator protection, reputation, performance tracking, dynamic commission, and stake concentration controls. The purpose is not only to select validators, but also to reduce the chance that a small group of operators or economic entities can dominate the chain over time. A validator should earn enough to run reliable infrastructure, but the protocol should avoid making excessive yield the central reason to participate.

The intended validator set is deliberately not tiny. A very small set is easier to coordinate, easier to pressure, and easier to capture. A very large set at launch, on the other hand, can create consensus overhead, weak operators, and difficult synchronization. Aetra aims for a middle path where the validator set grows only when operator readiness, network stability, and finality evidence justify the increase.

Nominator pools are included so that ordinary users do not have to manually choose a validator in order to participate in staking. A pool can aggregate stake, distribute shares, track reward indexes, process unbonding requests, and allocate stake across validators

according to policy. This reduces user complexity while still allowing the protocol to apply native slashing, validator-set rules, and concentration limits at the underlying consensus layer.

Anti-Concentration Policy

A central design rule is that large stake should not automatically compound into larger control. If rewards are strictly proportional to already-concentrated stake, the largest validators grow faster, delegators follow them because they appear safer, and the network slowly becomes easier to cartelize. Aetra therefore includes native stake-concentration and validator-score components. These components can measure top-N concentration, apply effective voting-power caps, warn against delegation into over-concentrated validators, and reduce the reward advantage of excess stake.

The same principle applies to commission. If validator commission is unbounded, operators can manipulate delegator behavior or race to unsustainable economics. If commission is forced too low, only large operators with outside subsidies can survive. The current architecture therefore includes commission floor, ceiling, and rate-change policy, plus performance and reputation modifiers. These rules are intended to keep validator income moderate, predictable, and tied to objective operation rather than to raw size alone.

Aetra Virtual Machine

The Aetra Virtual Machine, or AVM, is the native smart-contract environment. It is intended to support deploy and execute flows, external and internal messages, gas accounting, host-function limits, contract storage, receipts, events, and exit codes. AVM contracts are where user-defined logic should live. A developer should be able to create new on-chain entities, define their behavior, receive messages, send messages, store state, and compose higher-level applications without changing the protocol core.

This boundary is important. Protocol safety remains native; product and application logic moves to contracts. Staking, slashing, validator election, minting, burning, fee collection, treasury accounting, storage rent, and system configuration affect chain safety and supply correctness, so they remain built into the chain. User-defined contract logic can then evolve faster without forcing every application design into the core consensus surface.

Storage Rent and State Accountability

Aetra treats persistent state as a resource that must be accounted for. Contracts, account records, pool shares, reward indexes, unbonding records, and other long-lived state objects create storage pressure for every full node. Storage rent is therefore a protocol-level safety mechanism. Contracts with unpaid state costs may become frozen or limited according to explicit rules, while consensus-critical state must remain recoverable and governed by protocol-specific policies.

This design helps avoid unbounded state growth. It also connects to the medium-hardware goal: if state can grow forever without cost, running a node becomes harder over time, and decentralization declines. Storage rent is not merely an economic feature; it is part of keeping the network practical to verify.

System Entities

Aetra uses native system entities for protocol responsibilities that should not be ordinary user contracts. These include configuration, constitutional bounds, validator election, validator registry, treasury, fee collection, minting, burning, emissions, reputation, performance, scheduling, storage rent, identity root, bridge coordination, and future scaling

coordination. Some of these responsibilities are part of the immediate network core, while others are gated by testnet evidence and audit requirements.

The distinction between system entities and smart contracts gives the network a clearer trust model. Built-in entities protect consensus, supply, validator power, and chain configuration. AVM contracts provide programmability for applications. This separation is intended to make Aetra flexible without making the core protocol harder to audit.

Zones, Routing, and Future Scaling

Aetra's architecture includes the concept of execution zones, routing, load tracking, scheduling, and future shard coordination. These ideas point toward a future where the network can increase throughput by separating execution domains and routing messages deterministically. The early network, however, should not claim production sharding until simulator coverage, adversarial tests, export/import behavior, long-run testnet evidence, and independent audit are complete.

In the intended path, Aetra starts as a simpler BFT L1 with AVM support and strong validator economics. It can then add more advanced scheduling, zone commitments, cross-zone messages, and shard coordination only when those mechanisms are deterministic and safe. This keeps the first public network understandable while preserving a path to higher throughput.